

Week 1 – Day 2 Assignment

Assignment for Langflow

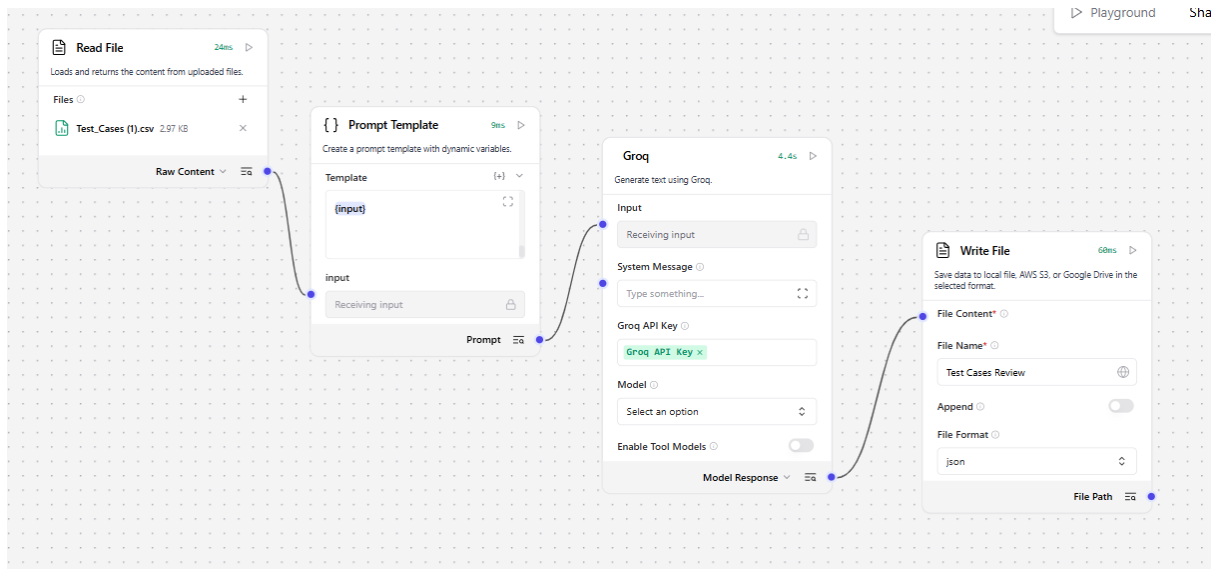
Build a test case reviewer using Langflow. Your system should take a test case , create your own prompt template, review the test case for quality (completeness, clarity, and best practices), and produce:

List of issues

Suggestions for improvement

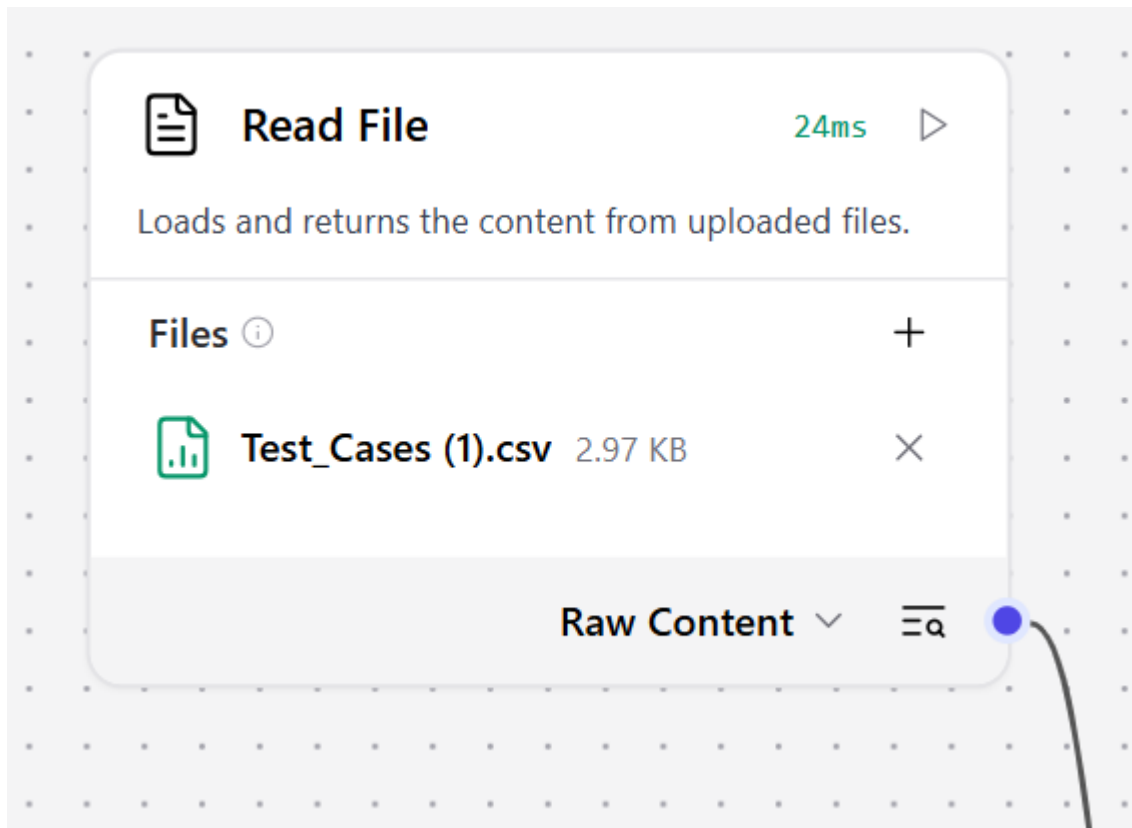
An improved version of the test case

Langflow – Complete flow



Step 1 :

Reading the test cases from csv file



Input File content :

Test Case ID	Test Scenario	Preconditions	Test Steps
TC_RSM_001	Verify successful stock addition for a new product	User is logged in with inventory management access	Navigate to Inventory Management 'Add New Product', Enter valid product name and SKU, Enter stock quantity, Save changes
TC_RSM_002	Verify duplicate SKU prevention	User is logged in with inventory management access	Navigate to Inventory Management 'Add New Product', Enter existing SKU, Save changes
TC_RSM_003	Verify negative stock prevention	User is logged in with inventory management access	Navigate to Inventory Management 'Update Stock', Enter negative quantity, Save changes
TC_RSM_004	Verify stock transfer between locations	User is logged in with inventory management access	Navigate to Inventory Management 'Transfer Stock', Select source and destination locations, Enter quantity, Save changes
TC_RSM_005	Verify stock update with invalid data	User is logged in with inventory management access	Navigate to Inventory Management 'Update Stock', Enter invalid data (e.g., non-numeric quantity), Save changes
TC_RSM_006	Verify role-based access for inventory management	User is logged in with limited access	Navigate to Inventory Management 'Add New Product'
TC_RSM_007	Verify stock reorder at minimum threshold	User is logged in with inventory management access	Navigate to Inventory Management 'Reorder Stock', Set reorder threshold, Trigger reorder

TC_RSM_008	Verify physical stock audit	User is logged in with inventory management access	Navigate to Inventory Management 'Physical Stock Audit', Perform audit
TC_RSM_009	Verify stock sync failure handling	User is logged in with inventory management access	Navigate to Inventory Management sync failure, Verify error handling
TC_RSM_010	Verify stock limit and threshold configuration	User is logged in with inventory management access	Navigate to Inventory Management stock limit and threshold, Verify configuration

Output

Component Output

Inspect the output of the component below.

Outputs

Logs

Test Case ID,Test Scenario,Preconditions,Test Steps,Test Data,Expected Result,Priority,Type

TC_RSM_001,Verify successful stock addition for a new product,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Add New Product', Enter valid product details and SKU, Enter stock quantity, Save the product","Product: 'Test Product', SKU: 'TEST123', Quantity: 10",Product is created and stock is updated successfully,High,Positive

TC_RSM_002,Verify duplicate SKU prevention,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Add New Product', Enter existing SKU, Save the product","Product: 'Test Product', SKU: 'EXISTING_SKU'",Error: 'Duplicate SKU found'",Medium,Validation

TC_RSM_003,Verify negative stock prevention,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Update Stock', Enter negative quantity, Save changes","Product: 'Test Product', Quantity: -5","Error: 'Negative stock not allowed'",High,Validation

TC_RSM_004,Verify stock transfer between locations,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Transfer Stock', Select source and destination locations, Enter quantity, Save transfer","Source Location: 'Store A', Destination Location: 'Warehouse', Quantity: 5",Stock is transferred successfully,Medium,Positive

TC_RSM_005,Verify stock update with invalid data,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Update Stock', Enter invalid data (e.g. non-numeric quantity), Save changes","Product: 'Test Product', Quantity: 'abc'",Error: 'Invalid data'",Medium,Validation

TC_RSM_006,Verify role-based access for inventory management,User is logged in with limited access,"Navigate to Inventory Management, Select 'Add New Product'",User Role: 'Sales'",Access denied,High,Authentication

TC_RSM_007,Verify stock reorder at minimum threshold,User is logged in with inventory management access,"Navigate to Inventory Management, Configure reorder threshold, Trigger reorder","Product: 'Test Product', Threshold: 5, Quantity: 5",Reorder triggered successfully,Medium,Positive

TC_RSM_008,Verify physical stock audit,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Physical Stock Audit', Perform audit","Product: 'Test Product', Quantity: 10",Audit completed successfully,Low,Positive

Close

Extracted content

Test Case ID,Test Scenario,Preconditions,Test Steps,Test Data,Expected Result,Priority,Type

TC_RSM_001,Verify successful stock addition for a new product,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Add New Product', Enter valid product details and SKU, Enter stock quantity, Save the product","Product: 'Test Product', SKU: 'TEST123', Quantity: 10",Product is created and stock is updated successfully,High,Positive

TC_RSM_002,Verify duplicate SKU prevention,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Add New Product', Enter existing SKU, Save the product","Product: 'Test Product', SKU: 'EXISTING_SKU'",Error: 'Duplicate SKU found'",Medium,Validation

TC_RSM_003,Verify negative stock prevention,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Update Stock', Enter negative quantity, Save changes","Product: 'Test Product', Quantity: -5","Error: 'Negative stock not allowed'",High,Validation

TC_RSM_004,Verify stock transfer between locations,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Transfer Stock', Select source

and destination locations, Enter quantity, Save transfer","Source Location: 'Store A', Destination Location: 'Warehouse', Quantity: 5",Stock is transferred successfully,Medium,Positive

TC_RSM_005,Verify stock update with invalid data,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Update Stock', Enter invalid data (e.g. non-numeric quantity), Save changes","Product: 'Test Product', Quantity: 'abc'","Error: 'Invalid data'",Medium,Validation

TC_RSM_006,Verify role-based access for inventory management,User is logged in with limited access,"Navigate to Inventory Management, Select 'Add New Product'","User Role: 'Sales'","Access denied,High,Authentication

TC_RSM_007,Verify stock reorder at minimum threshold,User is logged in with inventory management access,"Navigate to Inventory Management, Configure reorder threshold, Trigger reorder","Product: 'Test Product', Threshold: 5, Quantity: 5",Reorder triggered successfully,Medium,Positive

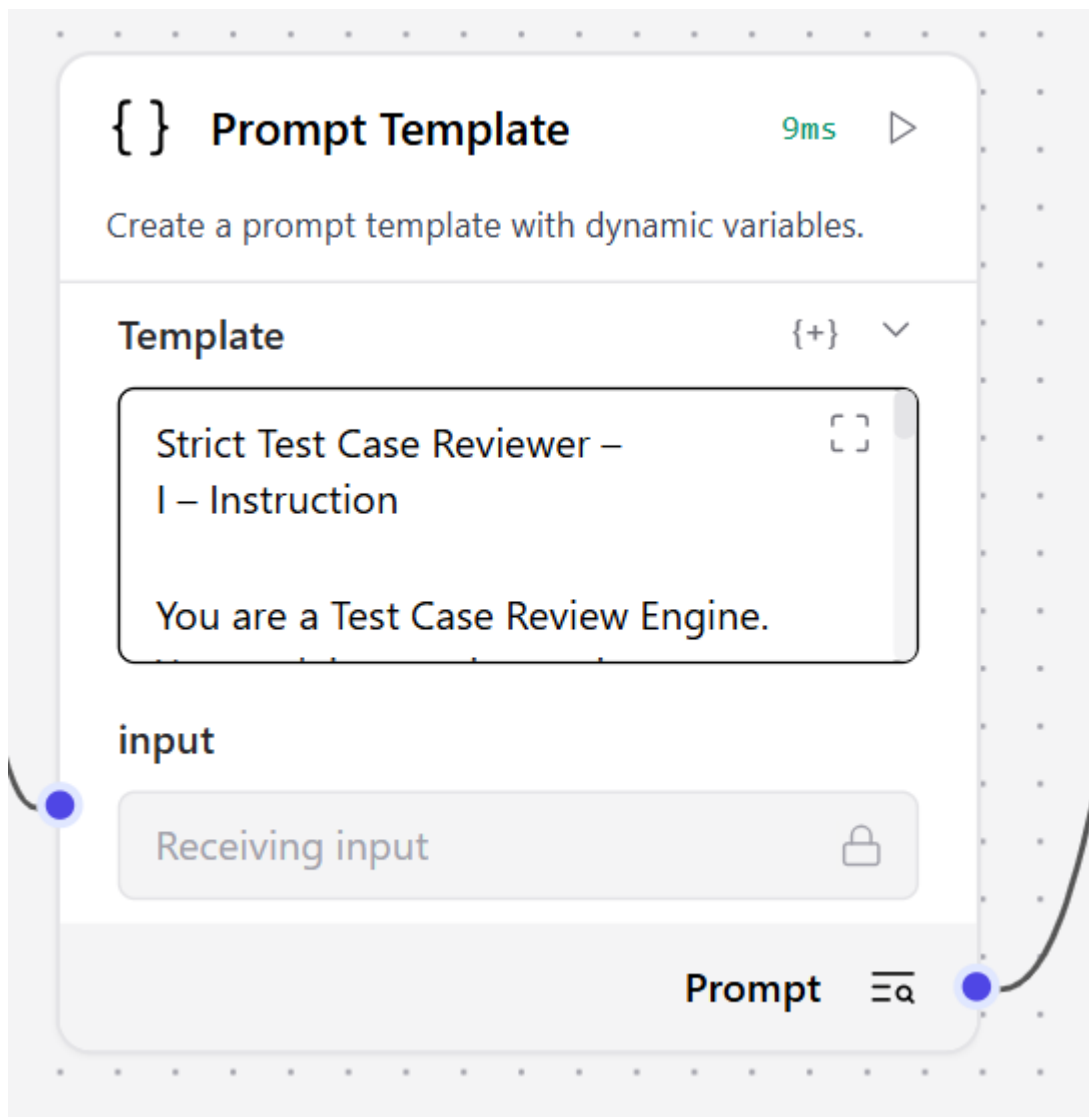
TC_RSM_008,Verify physical stock audit,User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Physical Stock Audit', Perform audit","Product: 'Test Product', Quantity: 10",Audit completed successfully,Low,Positive

TC_RSM_009,Verify stock sync failure handling,User is logged in with inventory management access,"Navigate to Inventory Management, Simulate sync failure, Verify error handling","Product: 'Test Product', Quantity: 10'","Error: 'Sync failure'",High>Error Handling

TC_RSM_010,Verify stock limit and threshold configuration,User is logged in with inventory management access,"Navigate to Inventory Management, Configure stock limit and threshold, Verify configuration","Product: 'Test Product', Limit: 100, Threshold: 20",Configuration saved successfully,Medium,Positive

Step 2 :

Prompting



Prompt Used

Strict Test Case Reviewer –

I – Instruction

You are a Test Case Review Engine.

Your task is to review a given test case ONLY when the user explicitly provides a test case for review.

You must evaluate the test case for:

Completeness

Clarity

Best Practices

Domain Validation

You must produce:

List of Issues

Suggestions for Improvement

Improved Version of the Test Case

CRITICAL RULES (Non-Negotiable):

If the input does NOT contain a test case, respond ONLY with:

"Out of scope: The request does not contain a test case for review."

Do NOT:

Generate new test cases unless improving the given one

Provide explanations outside the required output

Infer missing business context beyond what is necessary for correction

Check for unsafe or prohibited content (fraud scenarios, unauthorized access misuse, system abuse).

If unsafe, respond:

"Out of scope: The request contains unsafe or prohibited content."

Do NOT assume intent.

Do NOT expand scope.

C – Context

You are an AI Assistant with deep expertise in:

Functional Testing

Test Design Techniques (BVA, EP, Decision Tables)

Test Case Review & Audit

QA Best Practices & Standards

Enterprise Test Documentation

Domain Validation (Retail)

Focus on:

Missing validations

Weak expected results

Poorly defined steps

Incomplete test data

Lack of negative/boundary coverage

E – Example (Reference Format Only)

Test Case ID: TC_001

Title: Verify successful login

Preconditions: User is registered

Steps: Enter username and password → क्लिक login

Expected Result: Login successful

(Output should identify issues and improve this)

(Do NOT reuse this example unless explicitly applicable.)

O – Output Rules

ONLY when the input is a valid test case, produce output in the below structure:

Issues Identified:

Issue 1

Issue 2

Issue 3

Suggestions for Improvement:

Suggestion 1

Suggestion 2

Suggestion 3

Improved Test Case:

Test Case ID:

Title:

Preconditions:

Steps (numbered, atomic, detailed):

Test Data:

Expected Result:

Priority:

Type:

Rules:

Do NOT repeat the original test case

Improve clarity, add validations, and fix structure

Ensure steps are executable and unambiguous

Strengthen expected results with verifiable outcomes

Add missing test data if required

T – Tone

Professional

Structured

Precise

Enterprise QA–standard

Neutral and objective

{input}

Output :

Component Output

Inspect the output of the component below.

Outputs

Logs

Enterprise QA—standard

Neutral and objective

Test Case ID,Test Scenario,Preconditions,Test Steps,Test Data,Expected Result,Priority,Type
TC_RSM_001,Verify successful stock addition for a new product.User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Add New Product', Enter valid product details and SKU, Enter stock quantity, Save the product","Product: 'Test Product', SKU: 'TEST123', Quantity: 10",Product is created and stock is updated successfully,High,Positive
TC_RSM_002,Verify duplicate SKU prevention.User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Add New Product', Enter existing SKU, Save the product","Product: 'Test Product', SKU: 'EXISTING_SKU'",Error: 'Duplicate SKU found'",Medium,Validation
TC_RSM_003,Verify negative stock prevention.User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Update Stock', Enter negative quantity, Save changes","Product: 'Test Product', Quantity: -5",Error: 'Negative stock not allowed'",High,Validation
TC_RSM_004,Verify stock transfer between locations.User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Transfer Stock', Select source and destination locations, Enter quantity, Save transfer","Source Location: 'Store A', Destination Location: 'Warehouse', Quantity: 5",Stock is transferred successfully,Medium,Positive
TC_RSM_005,Verify stock update with invalid data.User is logged in with inventory management access,"Navigate to Inventory Management, Select 'Update Stock', Enter invalid data (e.g. non-numeric quantity), Save changes","Product: 'Test Product', Quantity: 'abc'",Error: 'Invalid data'",Medium,Validation
TC_RSM_006,Verify role-based access for inventory management.User is logged in with limited access,"Navigate to Inventory Management, Select 'Add New Product'",User Role: 'Sales',Access denied,High,Authentication
TC_RSM_007,Verify stock reorder at minimum threshold.User is logged in with inventory management access,"Navigate to Inventory Management, Configure reorder threshold, Trigger reorder","Product: 'Test Product', Threshold: 5, Quantity: 5",Reorder triggered successfully,Medium,Positive

Close

Step 3 : LLM

Groq6.8s

Generate text using Groq.

Input

Receiving input

System Message

Type something...

Groq API Key

Groq API Key x

Model

Select an option

Search options...

llama-3.1-8b-instant

llama-3.3-70b-versatile

openai/gpt-oss-120b

Output :

Component Output

Inspect the output of the component below.

OutputsLogs

Issues Identified

Test Case ID	Issue(s)
--------------	----------

TC_RSM_001	1. Steps are not atomic – multiple actions are combined in a single step. 2. No verification of the product details after creation (e.g., checking that the SKU, name, and quantity appear correctly in the product list). 3. Test Data column mixes description and values; should be a separate table. 4. Missing priority justification and type clarification (Functional vs. UI).
TC_RSM_002	1. "Enter existing SKU" does not specify which product already uses that SKU. 2. Expected result only mentions the error message; does not verify that the product is "not" created. 3. No pre-condition to ensure the SKU already exists in the system. 4. Steps are not numbered or atomic.
TC_RSM_003	1. "Select 'Update Stock'" – unclear whether this is from a product detail page or a bulk screen. 2. No verification that the stock quantity remains unchanged after the error. 3. Test Data mixes product name and quantity in one field. 4. Expected result does not specify UI behavior (e.g., inline validation vs. modal).
TC_RSM_004	1. No pre-condition that the source location actually has the required quantity. 2. Steps lack atomicity (source/destination selection and quantity entry are combined). 3. Expected result does not verify inventory levels at both locations after transfer. 4. Missing negative test (e.g., transfer quantity > available).
TC_RSM_005	1. "Enter invalid data (e.g. non-numeric quantity)" is vague – should specify the exact invalid value used. 2. No verification that the system prevents the save operation. 3. Expected result only mentions a generic error; does not state where the error is displayed.
TC_RSM_006	1. "User is logged in with limited access" – the role description is ambiguous; should specify the exact role (e.g., "Sales"). 2. Steps do not attempt to perform the action after navigation (e.g., click "Add New Product"). 3. Expected result only says "Access denied" without indicating UI message or HTTP status.
TC_RSM_007	1. No pre-condition that current stock equals the threshold. 2. "Trigger reorder" is not a user action – the system should automatically trigger when

Close

****Issues Identified****

| Test Case ID | Issue(s) |

|-----|-----|

| TC_RSM_001 | 1. Steps are not atomic – multiple actions are combined in a single step.
2. No verification of the product details after creation (e.g., checking that the SKU, name, and quantity appear correctly in the product list).
3. Test Data column mixes description and values; should be a separate table.
4. Missing priority justification and type clarification (Functional vs. UI). |

| TC_RSM_002 | 1. “Enter existing SKU” does not specify which product already uses that SKU.
2. Expected result only mentions the error message; does not verify that the product is ****not**** created.
3. No pre-condition to ensure the SKU already exists in the system.
4. Steps are not numbered or atomic. |

| TC_RSM_003 | 1. “Select ‘Update Stock’” – unclear whether this is from a product detail page or a bulk screen.
2. No verification that the stock quantity remains unchanged after the error.
3. Test Data mixes product name and quantity in one field.
4. Expected result does not specify UI behavior (e.g., inline validation vs. modal). |

| TC_RSM_004 | 1. No pre-condition that the source location actually has the required quantity.
2. Steps lack atomicity (source/destination selection and quantity entry are combined).
3. Expected result does not verify inventory levels at both locations after transfer.
4. Missing negative test (e.g., transfer quantity > available). |

| TC_RSM_005 | 1. “Enter invalid data (e.g. non-numeric quantity)” is vague – should specify the exact invalid value used.
2. No verification that the system prevents the save operation.
3. Expected result only mentions a generic error; does not state where the error is displayed. |

| TC_RSM_006 | 1. “User is logged in with limited access” – the role description is ambiguous; should specify the exact role (e.g., *Sales*).
2. Steps do not attempt to perform the action after navigation (e.g., click “Add New Product”).
3. Expected result only says “Access denied” without indicating UI message or HTTP status. |

| TC_RSM_007 | 1. No pre-condition that current stock equals the threshold.
2. “Trigger reorder” is not a user action – the system should automatically trigger when threshold is reached.
3. Expected result does not verify that a purchase order or reorder notification is generated. |

| TC_RSM_008 | 1. “Perform audit” is not defined – does not state what actions the auditor takes (count, adjust, confirm).
2. No verification of audit log entry.
3. Expected result is generic; should confirm that system records the audit outcome. |

| TC_RSM_009 | 1. “Simulate sync failure” lacks detail on how the failure is induced (e.g., network outage, mock service).
2. No verification of system state after the error (e.g., pending sync queue).
3. Expected result only shows an error message; does not confirm that no data loss occurs. |

| TC_RSM_010 | 1. Steps combine configuration and verification in one step.
2. No validation that the configured limit and threshold are enforced (e.g., trying to exceed the limit).
3. Expected result does not specify where the confirmation appears. |

****Suggestions for Improvement****

1. ****Make steps atomic and numbered**** – each user interaction (click, enter data, press Save) should be a separate step.
2. ****Explicitly define test data**** in a dedicated “Test Data” section with clear key-value pairs.
3. ****Add pre-conditions that guarantee required system state**** (e.g., existing SKU, sufficient stock, user role).
4. ****Strengthen expected results**** to include verifiable UI elements, database state checks, or API responses.
5. ****Include both positive and negative/edge scenarios**** where applicable (e.g., boundary quantities, invalid formats).
6. ****Specify priority rationale and test type**** (Functional, Validation, Security, etc.).
7. ****Document post-conditions or cleanup steps**** when test data may affect subsequent tests.
8. ****Use consistent terminology**** (e.g., “Add New Product” vs. “Create Product”) and avoid ambiguous language.

****Improved Test Cases****

Below are the revised versions of the original test cases, applying the suggestions above.

Test Case: TC_RSM_001 – Verify successful stock addition for a new product

****Preconditions****

- User is logged in with *Inventory Management* role.
- No product exists with SKU `TEST123`.

****Test Data****

Field	Value
Product Name	Test Product
SKU	TEST123
Quantity	10
Price	25.00
Category	Electronics

****Steps****

1. From the dashboard, navigate to ****Inventory Management**** → ****Add New Product****.
2. In the *Product Name* field, enter `Test Product`.
3. In the *SKU* field, enter `TEST123`.
4. In the *Quantity* field, enter `10`.
5. Fill any mandatory additional fields (e.g., Price, Category).
6. Click ****Save****.
7. Verify that a success toast/message “Product created successfully” appears.
8. Search for SKU `TEST123` in the product list and open the product details.
9. Confirm that the displayed *Product Name*, *SKU*, and *Quantity* match the test data.

****Expected Result****

- Product is created; success message displayed.
- Product list shows the new product with correct details.
- Database record exists with `SKU = TEST123` and `Quantity = 10`.

****Priority****: High

****Type****: Functional / Positive

Test Case: TC_RSM_002 – Verify duplicate SKU prevention

Preconditions

- User is logged in with *Inventory Management* role.
- A product with SKU `EXISTING\_SKU` already exists.

Test Data

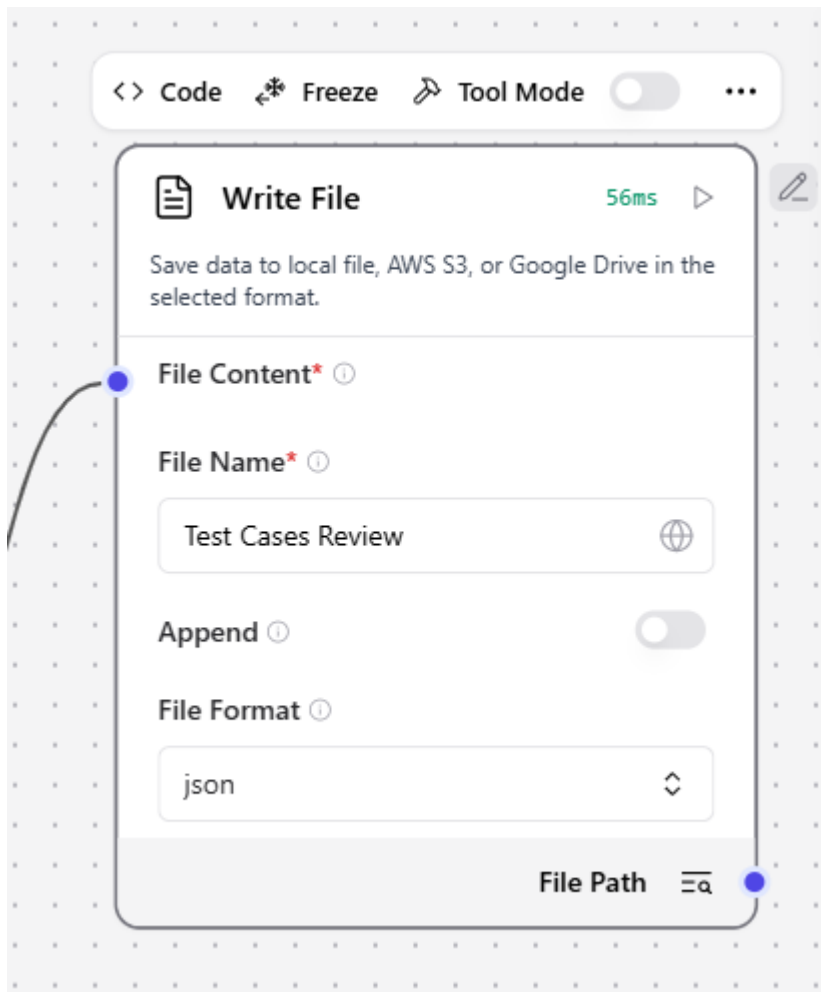
Field	Value
Product Name	Duplicate SKU Product
SKU	EXISTING_SKU
Quantity	5

Steps

1. Navigate to **Inventory Management** → **Add New Product**.
2. Enter `Duplicate SKU Product` in *Product Name*.
3. Enter `EXISTING\_SKU` in *SKU*.
4. Enter `5` in *Quantity*.
5. Click **Save**.

**Expected Result...

Step 4: Writing the output in json file



Output :

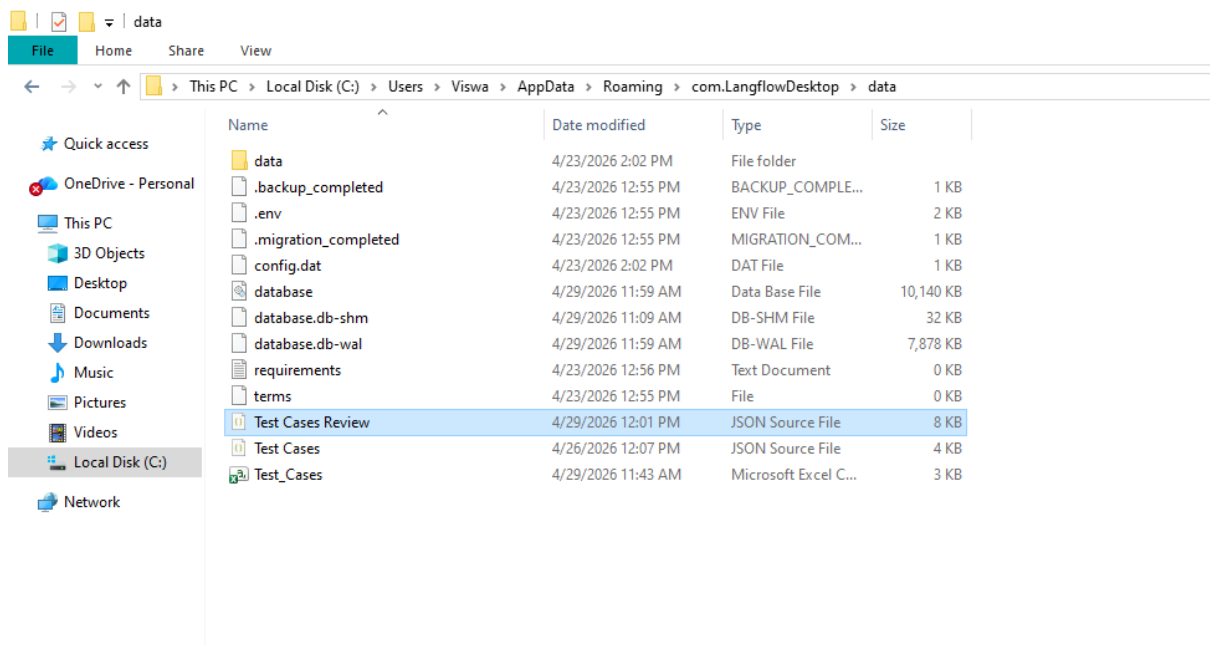
Component Output

Inspect the output of the component below.

Outputs Logs

Message saved successfully as 'Test Cases Review.json' at C:\Users\Viswa\AppData\Roaming\com.LangflowDesktop\data\Test Cases Review.json

Close



Content from json file

message: **Issues Identified**

| Test Case ID | Issue(s) |

|-----|-----|

| TC_RSM_001 | 1. Steps are not atomic – multiple actions combined in a single step.
2. No verification of UI messages or database state.
3. Test Data column mixes product name, SKU and quantity without clear format.
4. Missing priority justification and type clarification (functional vs. UI). |

| TC_RSM_002 | 1. “Enter existing SKU” does not specify which product the SKU belongs to.
2. Expected result only mentions error text; does not verify that the product is **not** created.
3. No negative test data for other required fields (e.g., missing product name). |

| TC_RSM_003 | 1. “Select ‘Update Stock’” – unclear whether this is from product detail page or a bulk screen.
2. Expected result only checks error message; should also confirm that stock level remains unchanged.
3. No pre-condition that the product already has a positive stock quantity. |

| TC_RSM_004 | 1. No validation that source location has sufficient quantity before transfer.
2. Steps lack confirmation step after “Save transfer”.
3. Expected result does not verify updated quantities at both locations. |

| TC_RSM_005 | 1. “Enter invalid data (e.g. non-numeric quantity)” is vague – should specify the exact invalid value used.
2. No check that the system prevents saving the record.
3. Missing verification that the original stock value is unchanged. |

| TC_RSM_006 | 1. Preconditions state “User is logged in with limited access” but test data shows a role of **Sales** – should be explicit that this role lacks inventory permissions.
2. Steps do not attempt to verify that the UI elements are hidden/disabled before trying to add a

product.
3. Expected result only says “Access denied” without specifying the message or UI behavior. |

| TC_RSM_007 | 1. No step to set the current stock level to the threshold value before triggering reorder.
2. Expected result does not verify that a reorder request is created (e.g., PO generated).
3. Missing validation of notification to the user. |

| TC_RSM_008 | 1. “Perform audit” is not defined – does not describe how the auditor records counted quantity.
2. No step to compare system quantity with physical count.
3. Expected result lacks detail on discrepancy handling. |

| TC_RSM_009 | 1. “Simulate sync failure” is not reproducible – needs a defined method (e.g., disable network, mock service).
2. Expected result only shows generic error; should verify that no partial data is persisted.
3. No rollback verification. |

| TC_RSM_010 | 1. Steps combine configuration and verification in one step.
2. Expected result does not confirm that the limits are enforced during subsequent stock operations.
3. No test data for edge values (e.g., quantity exactly at limit). |

****Suggestions for Improvement****

1. ****Make steps atomic and numbered**** – each user action should be a separate step; confirmations and validations should be explicit.
2. ****Add pre-conditions that set up required system state**** (e.g., existing product with known stock, user roles, location inventories).
3. ****Define test data in a structured table**** (field | value) and include both positive and negative values where applicable.
4. ****Strengthen expected results**** – include UI messages, database verification, and any side-effects (e.g., audit logs, PO creation).
5. ****Include priority justification**** and ensure the “Type” field matches standard taxonomy (Positive, Negative, Validation, Security, Performance, etc.).
6. ****Provide reproducible steps for error simulation**** (e.g., use a mock service, disable network, input out-of-range values).
7. ****Add boundary and negative cases**** where missing (e.g., zero quantity, maximum allowed quantity, special characters in SKU).
8. ****Document post-conditions**** where the test leaves the system (e.g., product created, stock unchanged).

****Improved Test Cases (selected examples)****

TC_RSM_001 – Verify successful stock addition for a new product

****Test Case ID:**** TC_RSM_001

****Title:**** Verify successful creation of a new product with initial stock

****Preconditions:****

- User is logged in with *Inventory Management* role.
- No existing product with SKU `TEST123` in the system.

****Test Data:****

Field	Value
Product Name	Test Product
SKU	TEST123
Initial Stock	10
Price	19.99
Category	Electronics

****Steps:****

1. Navigate to **Inventory Management** → **Add New Product**.
2. Enter **Product Name** = `Test Product`.
3. Enter **SKU** = `TEST123`.
4. Enter **Initial Stock** = `10`.
5. Fill required fields (Price, Category).
6. Click **Save**.
7. Verify that a success toast/message “Product created successfully” is displayed.

8. Query the database (or use UI search) to confirm the product exists with SKU `TEST123` and stock quantity `10`.

****Expected Result:****

- System shows a success message “Product created successfully”.
- The new product appears in the product list with the correct SKU and stock quantity `10`.
- Database record matches the entered data.

****Priority:**** High

****Type:**** Positive

TC_RSM_002 – Verify duplicate SKU prevention

****Test Case ID:**** TC_RSM_002

****Title:**** Prevent creation of a product with an existing SKU

****Preconditions:****

- User is logged in with *Inventory Management* role.
- Product with SKU `EXISTING\_SKU` already exists.

****Test Data:****

Field	Value
Product Name	Duplicate Test
SKU	EXISTING_SKU
Stock	5

****Steps:****

1. Navigate to **Inventory Management** → **Add New Product**.

2. Enter **Product Name** = ` Duplicate Test` .
3. Enter **SKU** = ` EXISTING\_SKU` .
4. Enter **Initial Stock** = ` 5` .
5. Click **Save**.
6. Observe the error message displayed.

Expected Result:

- System displays error “Duplicate SKU found”.
- No new product record is created; the existing product remains unchanged.

Priority: Medium

Type: Validation

TC_RSM_003 – Verify negative stock prevention

Test Case ID: TC_RSM_003

Title: Prevent updating stock with a negative quantity

Preconditions:

- User is logged in with *Inventory Management* role.
- Product ` Test Product ` (SKU ` TEST123 `) exists with current stock ` 10 ` .

Test Data:

Field	Value
SKU	TEST123
New Quantity	-5

Steps:

1. Navigate to **Inventory Management** → **Search Product** → enter SKU `TEST123`.
2. Open the product detail page and click **Update Stock**.
3. Enter **Quantity** = `-5`.
4. Click **Save**.
5. Verify the error message shown.
6. Refresh the product detail page and confirm the stock remains `10`.

Expected Result:

- System shows error “Negative stock not allowed”.
- Stock quantity for `TEST123` remains unchanged at `10`.

Priority: High

Type: Validation

(The remaining test cases – TC_RSM_004 through TC_RSM_010 – should be revised following the same pattern: clear pre-conditions, atomic steps, structured test data, and verifiable expected results. Apply the suggestions above to each.)